Helm ⭐ ⭐ — Interview-Ready Q&A

1. What is Helm?
Helm is a **package manager for Kubernetes** used to deploy, manage, upgrade, and rollback Kubernetes applications.
It uses reusable **Charts, Templates, and values** instead of maintaining many Kubernetes YAML files manually.
Simple ga gurthupettuko:
Helm = Kubernetes applications ni **package + deploy + upgrade + rollback** cheyyadaniki tool.
Eng: Helm is a package manager for Kubernetes that simplifies application deployment and management using reusable Charts and templates.

---

2. Why do we use Helm?
In real-time applications, we may have Deployment, Service, ConfigMap, Secret, Ingress, and other YAML files.
Dev, QA, and Prod environments ki same YAML ni different values tho maintain cheyyali. Helm templates and values files make this easier and reusable.
Simple ga gurthupettuko:
Many YAMLs → Helm Chart → Reusable Deployment
Eng: Helm is used to simplify Kubernetes deployments, reduce YAML duplication, support environment-specific configuration, and manage application releases.

---

3. What is a Helm Chart?
A Helm Chart is a **package of Kubernetes resources** required to deploy an application.
It contains files such as Chart.yaml, values.yaml, and template files.
Simple ga gurthupettuko:
Chart = Kubernetes application package.
Eng: A Helm Chart is a reusable package containing Kubernetes templates, configuration values, and metadata required to deploy an application.

---

4. What is the structure of a Helm Chart?
A basic Helm Chart structure looks like:

```
my-app/
├── Chart.yaml
├── values.yaml
├── charts/
└── templates/
    ├── deployment.yaml
    ├── service.yaml
    ├── configmap.yaml
    └── ingress.yaml
```

Chart.yaml contains metadata, values.yaml contains configuration values, and templates/ contains Kubernetes resource templates.
Simple ga gurthupettuko:
Chart.yaml = Metadata
values.yaml = Configuration
templates = Kubernetes YAML templates
Eng: A Helm Chart typically contains metadata, default values, optional dependencies, and Kubernetes resource templates.

---

5. What is Chart.yaml?
Chart.yaml is the main metadata file of a Helm Chart.
It contains information such as chart name, chart version, description, and application version.
Simple ga gurthupettuko:
Chart.yaml = Chart information.
Eng: Chart.yaml defines the metadata and version information of a Helm Chart.

---

6. What is values.yaml?
values.yaml contains the **default configuration values** used by Helm templates.
For example, we can define image repository, image tag, replica count, service type, ports, and resource settings.
Simple ga gurthupettuko:
values.yaml = Application configuration values.
Eng: values.yaml provides default configuration values that are consumed by Helm templates and can be overridden for different environments.

---

7. What are Helm Templates?
Helm Templates are Kubernetes YAML files containing dynamic expressions.
Instead of hardcoding values, we reference values from values.yaml. Helm processes these templates and generates the final Kubernetes manifests.
Simple ga gurthupettuko:
Template + Values → Final Kubernetes YAML
Eng: Helm templates are parameterized Kubernetes manifests that are rendered using supplied values to generate the final resources.

8. What is a Helm Release?
A Helm Release is a **deployed instance of a Helm Chart**.
For example, if we install a chart with release name payment-app, Helm tracks that deployment as the payment-app release.
Simple ga gurthupettuko:
Chart = Package
Release = Installed/deployed package
Eng: A Helm Release is a specific deployed instance of a Helm Chart in a Kubernetes cluster, with its configuration and revision history managed by Helm.

---

9. What is the difference between Chart and Release?
A **Chart** is the reusable package.
A **Release** is the actual deployed instance of that Chart.
The same Chart can be deployed multiple times with different release names or configurations.
Simple ga gurthupettuko:
Chart = What to deploy
Release = Deployed instance
Eng: A Chart is a reusable Kubernetes application package, while a Release is an installed instance of that Chart.

---

10. How do you install a Helm Chart?
We use:
helm install payment-app ./my-chart
Helm renders the templates using the configured values and creates the Kubernetes resources in the cluster.
Simple ga gurthupettuko:
helm install = Chart ni Kubernetes lo deploy cheyyadam.
Eng: helm install creates a new Helm Release by rendering the Chart templates and deploying the resulting Kubernetes resources.

---

11. How do you install a Helm Chart with a custom values file?
We can use the -f option.
helm install payment-app ./my-chart -f values-prod.yaml
This allows us to use the same Chart with Production-specific configuration.
Simple ga gurthupettuko:
One Chart + values-prod.yaml = Production deployment.
Eng: A custom values file allows us to override default configuration and deploy the same Helm Chart with environment-specific settings.

---

12. How do you upgrade a Helm Release?
We use:
helm upgrade payment-app ./my-chart
During the upgrade, Helm applies the new configuration and updates the Kubernetes resources managed by that release.
Simple ga gurthupettuko:
Old Release → New configuration → Upgrade
Eng: helm upgrade updates an existing Helm Release using a new Chart version, configuration, or application image.

---

13. What is Helm Rollback?
Rollback is used to return a Helm Release to a previous revision when the current deployment has problems.
First we check:
helm history payment-app
Then:
helm rollback payment-app 2
Simple ga gurthupettuko:
New deployment problem → Previous stable revision → Rollback
Eng: Helm rollback restores a release to a previous revision, which is useful when a new deployment causes application or configuration issues.

---

14. What is Helm Revision?
Every install or upgrade creates a revision for the Helm Release.
For example:
Revision 1 → Initial deployment
Revision 2 → Application v2
Revision 3 → Application v3
These revisions help us understand deployment history and perform rollbacks.
Simple ga gurthupettuko:
Revision = Release deployment history.
Eng: A Helm revision represents a version of a Release and allows us to track changes and roll back to an earlier deployment.

---

15. What is a Helm Repository?
A Helm Repository is a location where Helm Charts are stored and distributed.
Teams can use public repositories or private repositories to store and share reusable Charts.
Simple ga gurthupettuko:
Helm Repository = Charts store/share chese place.
Eng: A Helm Repository is a location used to store and distribute Helm Charts so they can be searched, downloaded, and installed.

16. What is helm repo add**?**
helm repo add is used to add a Helm Repository to the local Helm configuration.
Example:
helm repo add myrepo <repository-url>
After adding it, we can search and install Charts from that repository.
Simple ga gurthupettuko:
repo add = Helm ki repository add cheyyadam.
Eng: helm repo add registers a Helm Chart repository with the Helm client so Charts from that repository can be accessed.

---

17. What is helm repo update**?**
helm repo update refreshes the local metadata of configured Helm repositories.
It helps Helm discover newly published or updated Chart versions.
Simple ga gurthupettuko:
repo update = Latest Chart information refresh.
Eng: helm repo update refreshes repository metadata so the Helm client can discover the latest available Charts and versions.

---

18. What is helm list**?**
helm list shows the Helm Releases installed in the current namespace.
Example:
helm list
For another namespace:
helm list -n production
Simple ga gurthupettuko:
helm list = Installed releases check.
Eng: helm list is used to view Helm Releases and their current deployment information within a namespace.

---

19. What is helm status**?**
helm status provides information about a particular Helm Release.
Example:
helm status payment-app
It is useful during troubleshooting to understand whether the release is deployed successfully and what resources are associated with it.
Simple ga gurthupettuko:
helm status = Release current status check.
Eng: helm status displays the current status and information of a Helm Release and is commonly used during deployment troubleshooting.

---

20. What is helm history**?**
helm history shows the revision history of a Helm Release.
Example:
helm history payment-app
It helps us identify previous versions and decide which revision should be restored during a rollback.
Simple ga gurthupettuko:
helm history = Previous deployment versions.
Eng: helm history displays the revision history of a Helm Release and is useful for troubleshooting and rollback operations.

---

21. What is helm template**?**
helm template renders the Helm templates locally and displays the final Kubernetes YAML.
It does **not deploy** anything to the cluster.
This is very useful for validating templates and values before deployment.
Simple ga gurthupettuko:
helm template = Final YAML preview.
Eng: helm template renders Helm templates locally so we can inspect the generated Kubernetes manifests before deploying them.

---

22. Difference between helm template **and** helm install**?**
helm template only generates the Kubernetes YAML.
helm install generates the YAML and actually creates the resources in the Kubernetes cluster.
Simple ga gurthupettuko:
Template = Preview
Install = Actual deployment
Eng: helm template is used for rendering and validation, while helm install renders the Chart and deploys the resulting resources into Kubernetes.

---

23. What is helm lint**?**
helm lint checks a Helm Chart for common errors and potential problems.
Example:
helm lint ./my-chart
It can be included as a validation step before deployment.
Simple ga gurthupettuko:
helm lint = Chart basic validation.
Eng: helm lint validates a Helm Chart and identifies common structural or configuration issues before deployment.

24. How can you override Helm values?
We can override values using another values file:
helm upgrade payment-app ./chart -f values-prod.yaml
We can also use --set for individual values:
helm upgrade payment-app ./chart --set image.tag=2.0
Simple ga gurthupettuko:
-f = Values file
--set = Individual value override
Eng: Helm values can be overridden using additional values files or the --set option, allowing environment-specific configuration without modifying templates.

---

25. How do you manage Dev, QA, and Production using Helm?
Usually, we maintain **one common Chart** and separate values files.
Helm Chart
├── values-dev.yaml
├── values-qa.yaml
└── values-prod.yaml
The CI/CD pipeline selects the appropriate values file based on the deployment environment.
Simple ga gurthupettuko:
One Chart + Different Values = Dev / QA / Prod
Eng: I use a common reusable Helm Chart with environment-specific values files to maintain consistency while allowing different configurations for Dev, QA, and Production.

---

26. How do you use Helm in CI/CD?
A typical flow is:
Developer
↓
Git
↓
Jenkins / CI Pipeline
↓
Build Docker Image
↓
Push to Registry
↓
Helm Upgrade
↓
Kubernetes
The pipeline passes the new Docker image tag to Helm and deploys it to the required environment.
Simple ga gurthupettuko:
CI = Image build
Helm = Kubernetes deployment
Eng: In CI/CD, Helm is commonly used in the deployment stage to update Kubernetes applications with the new container image and environment-specific configuration.

---

27. What happens when you run helm upgrade **with a new image tag?**
Helm takes the new value, renders the Deployment template, and updates the Kubernetes Deployment.
Kubernetes then performs the configured rollout, creating new Pods with the new image and replacing old Pods according to the deployment strategy.
Simple ga gurthupettuko:
New Image Tag → Helm Upgrade → Deployment Update → New Pods
Eng: During a Helm upgrade with a new image tag, Helm updates the Kubernetes manifests and Kubernetes performs the application rollout according to the Deployment strategy.

---

28. How do you troubleshoot a failed Helm deployment?
First I check:
helm status <release>
helm history <release>
Then I check the rendered YAML using helm template and inspect Kubernetes resources:
kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name>
I check image issues, configuration, Secrets, probes, resources, and Kubernetes events.
Simple ga gurthupettuko:
Helm Status → History → Template → Pods → Events → Logs → Root Cause
Eng: I troubleshoot a failed Helm deployment by checking release status and history, validating rendered manifests, and then investigating Kubernetes events, Pod status, images, configuration, probes, and application logs.

29. How do you perform a rollback in Production?
First I identify the current and previous revisions:
helm history payment-app
Then I select the last known-good revision and execute:
helm rollback payment-app <revision>
After rollback, I verify Pods, application health, logs, and traffic to confirm that the previous version is working correctly.
Simple ga gurthupettuko:
History → Previous Stable Revision → Rollback → Verify
Eng: In Production, I use Helm revision history to identify the last stable release, perform a controlled rollback, and then verify application health and traffic.

---

30. Explain a real-time Helm deployment flow.
Developer code ni Git lo push chestadu.
CI pipeline application ni build chesi Docker image create chestundi and container registry ki push chestundi. CD pipeline new image tag ni Helm ki pass chesi helm upgrade execute chestundi.
Helm Kubernetes manifests render chesi Deployment update chestundi. Kubernetes new Pods create chesi readiness checks pass ayyaka traffic serve chestundi.
Simple ga gurthupettuko:
Git
↓
Jenkins / CI
↓
Docker Build
↓
Container Registry
↓
Helm Upgrade
↓
Kubernetes Deployment
↓
New Pods
↓
Readiness Check
↓
Application Traffic
Eng: In a real-time DevOps workflow, CI builds and pushes the container image, while the CD pipeline uses Helm to deploy the new image to Kubernetes. Kubernetes performs the rollout and health checks, and Helm provides release history and rollback capability if the deployment fails.